TMC, MP-RWS, MPIW, the Source Term Trick, and QEM

How `alan` does inference, plus a look at some newer empirical follow-ups

Outline

The problem

Why one sample per latent isn't enough

- Standard importance sampling / VI: draw *one* sample of the whole latent vector z from Q, reweight by $p(z, x)/q(z)$.
- Better: draw K *whole-vector* samples (IWAE-style) and average: tighter bound, lower variance, but cost is $O(K)$ per *full model*, and you need K large before it helps on models with many latents.
- What if we could afford K samples *per latent variable*, not per whole vector? Naively that's $K^{\#\text{latents}}$ combinations to consider: looks hopeless.

This is the problem Tensor Monte Carlo, then Massively-Parallel RWS, and then the source term trick, solve in turn.

Wake-Sleep, briefly

Classic wake-sleep (Hinton et al.) trains a generative model P and a recognition/proposal model Q together:

Reweighted Wake-Sleep (RWS) improves the Q update: instead of training Q on P's own samples, use self-normalized importance weights from K samples of Q to push Q toward the *true posterior* directly. Unlike VI, RWS never reparameterizes: it works directly with discrete latent variables, no REINFORCE or relaxations needed.

Tensor Monte Carlo

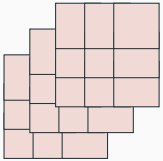
Tensor Monte Carlo (Aitchison, 2019)

The key move that makes "K samples per latent" tractable:

- Draw K candidate values for *each* latent variable separately, not K whole-vector samples.
- Most latents only interact with a handful of neighbours in the model's dependency graph (plates, conditional independence).
- So the $K^{\#\text{latents}}$ sum factorizes into a sequence of small tensor contractions (like `einsum`) that follow the graph structure: cost becomes polynomial in K, not exponential.

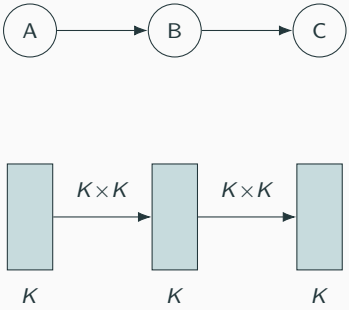
Why this matters: a 3-latent chain, $K = 3$

naive: all $K^3 = 27$ combinations



K slices of $K \times K$

structured: contract one edge at a time



$2K^2 = 18$ multiply-adds, not 27

$$\hat{P}_{\mathrm{MP}} = \sum_{k_a,k_b,k_c} f_A(k_a)\, f_{AB}(k_a,k_b)\, f_{BC}(k_b,k_c) = \sum_{k_c} \Big[\underbrace{\sum_{k_b}}_{\text{contract } B} v_B[k_b]\, f_{BC}(k_b,k_c) \Big]$$

where $v_B[k_b] = \sum_{k_a} f_A(k_a)\, f_{AB}(k_a,k_b)$ is computed once and reused for every k_c.

What TMC gives you (and doesn't)

+ A much lower-variance, K-per-latent marginal likelihood estimator, at polynomial cost.
– On its own, it's just an estimator: no prescription for *training* P and Q together, and no cheap way to get posterior samples/moments back out afterward.

Massively-Parallel Reweighted Wake-Sleep (Heap et al., UAI 2023) turns this estimator into a full training algorithm.

Massively-Parallel RWS

The estimator: $P_{\mathsf{MP}}(z)$

For each latent, draw K particles from Q; contract following the plate structure (as above) to get a single scalar:

$$\hat{P}_{\mathsf{MP}}(x) \;=\; \text{(TMC-style contraction of } p(z,x)/q(z) \text{ over all latents' } K \text{ particles)}$$

- Unbiased: $\mathbb{E}[\hat{P}_{\mathsf{MP}}(x)] = p(x)$, the true marginal likelihood, proved directly, not just assumed.
- This single fact is what later makes *exact* MCMC schemes (Particle MCMC) possible on top of this machinery; more on that at the end.

Two updates, one estimator

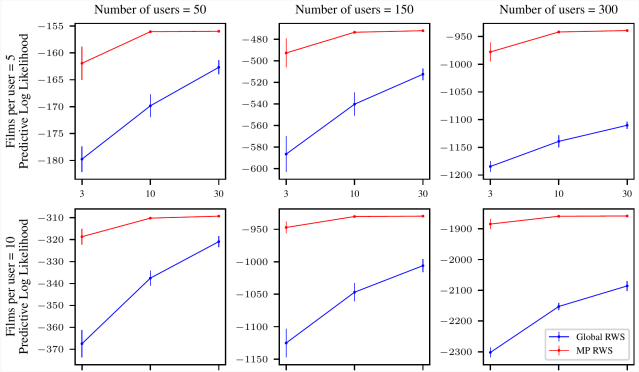
Practical form of the massively-parallel RWS updates (Heap et al., Eq. 19):

$$\Delta\theta_{\mathrm{MP}} = \mathbb{E}_{Q_{\mathrm{MP}}(z|x)}\big[\nabla_\theta \log \hat{P}_{\mathrm{MP}}(z)\big] \qquad \Delta\phi_{\mathrm{MP}} = \mathbb{E}_{Q_{\mathrm{MP}}(z|x)}\big[\nabla_\phi\big(-\log \hat{P}_{\mathrm{MP}}(z)\big)\big]$$

- **Wake phase**: ascend $\Delta\theta_{\mathrm{MP}}$, i.e. climb $\log \hat{P}_{\mathrm{MP}}$ w.r.t. P's parameters.
- **Sleep phase**: ascend $\Delta\phi_{\mathrm{MP}}$, i.e. *descend* $\log \hat{P}_{\mathrm{MP}}$ w.r.t. Q's parameters, which pushes Q toward the true posterior.

Equivalent to the fuller self-normalized-importance-weight form (Eq. 18): a weighted average of $\nabla \log P_\theta(z^{\mathbf{k}}, x)$ / $\nabla \log Q_\phi(z^{\mathbf{k}}, x)$ over all K^n combinations, weighted by $r_{\mathbf{k}}(z)/\hat{P}_{\mathrm{MP}}(z)$. Both forms reuse the same $\hat{P}_{\mathrm{MP}}(z)$ computation; no separate machinery for P vs. Q.

Results: MP RWS vs. global RWS, MovieLens



Predictive log-likelihood vs. $K \in \{3, 10, 30\}$, at three dataset sizes. MP RWS (red) beats global RWS (blue) at every K and every size, and the gap widens as the number of users grows (Heap et al., 2023).

MPIW

From "global" to massively-parallel importance weighting

MP-RWS trains P and Q. **MPIW** (Bowyer, Heap & Aitchison, UAI 2024) asks a different question: once trained, how do we get *posterior expectations* back out?

- **Global** importance weighting: draw K whole-vector samples, reweight, average: the classical scheme.
- **Massively-parallel** importance weighting (MPIW): draw K *per latent*, reweight *all* $K^{\#\text{latents}}$ combinations (same trick as before), and use *that* as the importance-sampling estimator of a posterior expectation.

This is the same $\hat{P}_{\text{MP}}(z)$ construction as before, now reframed as a general-purpose importance-weighting tool in its own right, not just something living inside the RWS training loop.

The math

With $\mathbf{k} = (k_1, \ldots, k_n)$ indexing one combination of particles (one per latent), and $\mathrm{qa}(i)$ the parents of latent i under Q:

$$r_{\mathbf{k}}(z) = \frac{P(x, z^{\mathbf{k}})}{\prod_i Q_{\mathrm{MP}}\left(z_i^{k_i} \mid z_j \text{ for } j \in \mathrm{qa}(i)\right)} \qquad \hat{P}_{\mathrm{MP}}(z) = \frac{1}{K^n} \sum_{\mathbf{k} \in \mathcal{K}^n} r_{\mathbf{k}}(z)$$

$r_{\mathbf{k}}(z)$ factorizes into low-rank tensors along the graph (as on the TMC slide), so this sum is computed via `opt_einsum`, never by looping over all K^n terms directly, giving one scalar, $\hat{P}_{\mathrm{MP}}(z)$, per forward pass.

The source term trick

Why can't we just calculate moments like that?

A posterior moment estimate looks like a similarly simple sum:

$$\hat{m}_{\mathrm{MP}}(z) = \frac{1}{K^n} \sum_{\mathbf{k} \in \mathcal{K}^n} \frac{r_{\mathbf{k}}(z)}{\hat{P}_{\mathrm{MP}}(z)} \, m(z^{\mathbf{k}})$$

- $\hat{P}_{\mathrm{MP}}(z)$ is one *forward* pass along the graph (contract, get a scalar): the TMC/MPIW trick handles that.
- But $\hat{m}_{\mathrm{MP}}(z)$ needs, for *each individual particle*, how much it contributed to that total. That means a backward pass through the same contraction, like backprop through the sum-product graph.
- That backward pass is genuinely complex, and different for every quantity: a sample, a marginal, a mean, a covariance each need their own hand-written traversal, for every plate topology.

The math of the source term trick

Add a source term J into the log joint, forming a modified normalizing constant (Bowyer, Heap & Aitchison, UAI 2024):

$$Z_m(J) = \int dz'\, P(x, z')\, e^{J\, m(z')} \qquad \Rightarrow \qquad \frac{\partial}{\partial J} \log Z_m(J)\bigg|_{J=0} = \mathbb{E}_{P(z'|x)}[m(z')]$$

The massively-parallel analogue replaces the integral with the same contraction as before, now carrying J through it:

$$\hat{P}^{\exp}_{\mathrm{MP}}(z, J) = \frac{1}{K^n} \sum_{\mathbf{k}} r_{\mathbf{k}}(z)\, e^{J\, m(z^{\mathbf{k}})} \qquad \Rightarrow \qquad \frac{\partial}{\partial J} \log \hat{P}^{\exp}_{\mathrm{MP}}(z, J)\bigg|_{J=0} = \hat{m}_{\mathrm{MP}}(z)$$

$J = 0$ recovers $\hat{P}_{\mathrm{MP}}(z)$ exactly, so this costs one extra forward pass plus one ordinary autodiff call, not a hand-written backward traversal.

Why this is the elegant part

- No new algorithm per quantity: `alan` computes posterior samples, marginals, *and* moments through literally the same code path that computes the bound.
- Works uniformly across arbitrarily nested plates: the contraction already handles that; the source term rides along for free.
- It's autodiff doing the bookkeeping that would otherwise be a hand-written, model-specific resampling routine.

The shape of a source-term-trick call

```
J: zeros, same shape as z_i (one entry per particle)
L_of_J = log P_MP_exp(z, J)        # scalar, same forward pass as before
m_MP = grad(L_of_J, J)             # same shape as J
```

- Input: a zero tensor J shaped like whichever latent's particles you want a quantity for. Output: a tensor the *same shape*, holding one number per particle.
- One `grad` call replaces a bespoke backward traversal; samples, marginals, and moments all use this exact shape, only $m(\cdot)$ (baked into $L(J)$) changes.

This is really in the code

From `alan`'s `Sample.py` (lightly trimmed):

```
J_tensor = zeros(shape, requires_grad=True)
L = self._elbo(sample, extra_log_factors={...:  J_tensor})
marginals = grad(L, J_tensor_list)
```

- `J_tensor` starts at all zeros: it doesn't change what `L` evaluates to.
- But differentiating `L` through it gives back exact per-particle marginal weights, for whichever latent(s) `J` was attached to.

QEM

Training Q still needs gradients

MP-VI and MP-RWS both update Q's parameters ϕ by **gradient ascent** on a massively-parallel bound.

- Needs a learning rate, tuned per model.
- Not invariant to reparameterization: rescale a latent by a constant (leaving the actual distribution over data untouched) and gradient descent's behaviour changes, and sometimes it stops converging at all.

QEM (Heap, Bowyer & Aitchison, 2025) removes gradient descent from the Q update entirely.

The QEM idea

E-step
Use MPIW + the source term trick to compute the *current* posterior moments, exactly as before.

M-step
Set Q's *next* parameters directly from those moments, in closed form, for exponential-family Q (Gaussian, Gamma, Beta, Dirichlet, Binomial, ...).

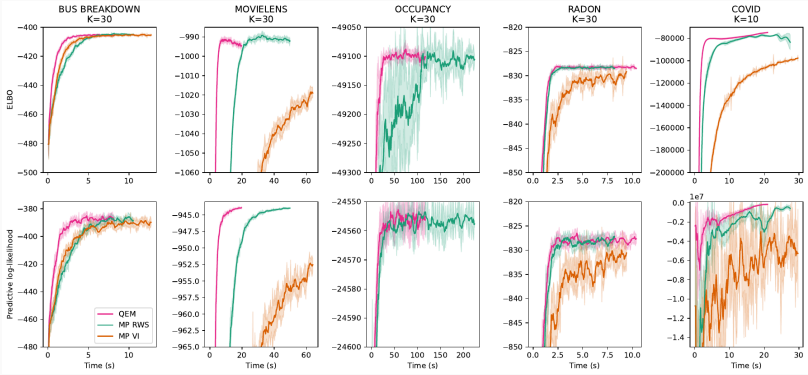
The name: it resembles EM, but optimizes the parameters of the *approximate posterior* Q, not the model's prior/likelihood parameters θ, as classical EM does. Hence "QEM."

Algorithm

```
for t in 1..T:
  Q_t = set_exp_family_params(m[t-1])  # M-step
  z = sample(Q_t)
  m_new = MPIW_moments(z, Q_t)     # E-step
  m[t] = lambda*m_new + (1-lambda)*m[t-1] # smoothing
```

- No gradients, no learning rate: just moment matching.
- The exponential moving average on the last line matters: a single noisy moment estimate can otherwise cause pathological collapse (e.g. near-zero effective weight on all but one sample). Provably converges to the unbiased, zero-variance estimate as $T \to \infty$.

Results: five real hierarchical models, ELBO vs. time



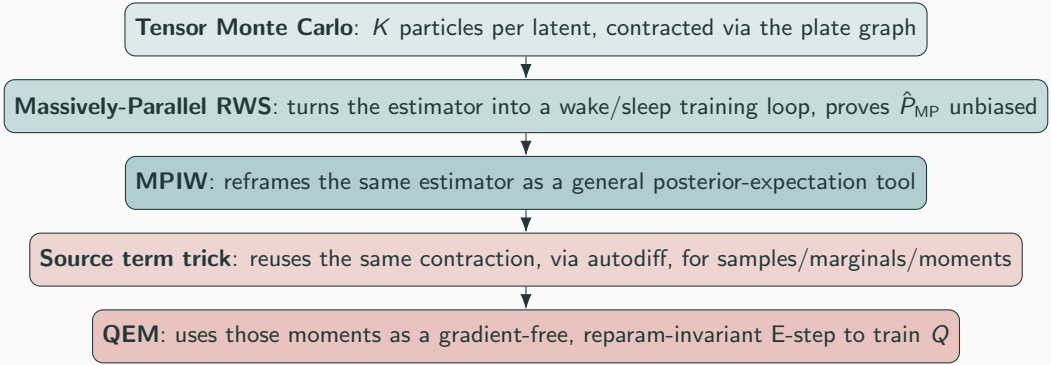
QEM (pink) reaches the same ELBO / predictive log-likelihood as MP-RWS (green) and MP-VI (orange) in a fraction of the wall-clock time, on Bus Breakdown, MovieLens100K, Occupancy, Radon, and Covid (Heap, Bowyer & Aitchison, 2025). No backprop through the whole joint per step.

Also: exactly reparameterization invariant

- **At least as accurate**: posterior first-moment MSE (vs. long HMC runs) at least as good as MP-RWS, and clearly better than MP-VI, throughout.
- Reparameterization invariant, exactly: rescale a latent by $\times 1/100$ (then back), and QEM's trajectory is unchanged. MP-VI *fails to converge at all within 250 iterations* on the reparameterized Radon and Covid models: a real, measured failure, not a hypothetical one.

How it fits together

The whole picture



Further work

Further work: ideas building on $\hat{P}_{\mathrm{MP}}$'s unbiasedness

- **How biased is MP-IS in practice, and does it actually close the gap to HMC?** An empirical open question worth measuring directly on a model with a known answer.
- **Better debiasing than jackknife**: chain-recycling schemes (BR-SNIS-style) that target the already plate-reduced estimator instead of the raw joint.
- **Exact inference via Particle Gibbs**: $\hat{P}_{\mathrm{MP}}$'s unbiasedness is exactly the ingredient Particle MCMC needs; pin a reference trajectory and get an exact posterior, not an approximate one.
- **Ancestor sampling (PGAS)** as the likely key to making Particle Gibbs mix well, given the plate-wise ESS unevenness the source term trick already lets us measure.

Wrap-up

Summary

- TMC makes K-per-latent tractable by contracting along the plate graph.
- MP-RWS turns that estimator into a training algorithm, and proves $\hat{P}_{\mathrm{MP}}$ unbiased.
- MPIW reframes the same estimator as a general tool for posterior expectations, not just RWS training.
- The source term trick gets samples/marginals/moments out of the *same* computation, via autodiff, for free.
- QEM closes the loop: uses those moments as a gradient-free, reparameterization-invariant E-step, beating MP-VI and MP-RWS on both speed and (at worst) matching them on accuracy.
- That whole stack opens up further work: measuring MP-IS bias in practice, better debiasing, and exact inference via Particle Gibbs, all building directly on $\hat{P}_{\mathrm{MP}}$'s unbiasedness.

Questions